

Workspace 'Week 2' in '6.006'

Page 1 (row 1, column 1)

Set Interface (L03-L08)

$A_1, A_2, \dots, E$

Set Interface

- Reductive: combines the input array
- In place: uses $O(1)$ extra space.

Input: Array of n numbers/keys A
Output: Sorted array B

Permutation Sort

1. Generate Permutation $S(n)$

2. def if permutation is sorted for $i=1$ to $n-1$ $O(n^2)$
 $B[i] \leq B[i+1]$

$\Rightarrow O(n! \cdot n)$

Selection Sort

1. Find min_index with $\text{index} \leq i$.
2. swap
3. $i = i + 1$

Find min_index from $0, \dots, i$

1. If i is min_index
2. If $\text{min_index} < i$

$8 \ 3 \ 5 \ 7 \ 9$

$8 \leftrightarrow 3$

```
def prefix_max(A, i):  
    """Return index of maximum in A[i:i+1].  
    if i > 0:  
        j = prefix_max(A, i-1)  
        if A[i] < A[j]:  
            return j  
    return i
```

$i = 0$
 $\Rightarrow$ $O(1)$

min_index

$g(i) = O(1)$
 $g(n) = g(n-1) + O(1)$
 $g(n) = O(n)$
 $= O(n) + O(1)$

$T(n) = T(n-1) + O(n) = O(n^2)$

$\checkmark$ $O(n^2)$ is $O(n^2)$

$T(n) = O(n^2)$

$O(n^2) = O(n^2) + O(n) = O(n^2) - 2O(n) + O(n)$

$O(n) = 2O(n) - O(n)$

Merge Sort

1 5 6 7
2 3 4 9

$\Rightarrow$ Merge sort is a divide and conquer algorithm.
It can be used to sort arrays.

```
def merge_sort(A, a, b, B = None):  
    """Sort A[a:b] into B.  
    if b is None: b = len(A)  
    if a < b-1:  
        m = (a+b)//2  
        merge_sort(A, a, m, B)  
        merge_sort(A, m, b, B)  
        merge(A[a:m], A[m:b], B)
```

$1 \ 3 \ 5 \ 7$
 $2 \ 4 \ 6 \ 8$
 $1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8$

$g(n) = g(n/2) + O(n)$

$\Rightarrow O(n \log n)$

Sorting

0 1 2 3 4 5
2 3 4 5 6 7

Can binary search.

Permutation
Sorted array

$\Rightarrow B[i-1] \leq B[i]$
 $\Rightarrow$ B is sorted.

Permutation Sort

Can generate permutation

Ex) $[2, 3, 1] \rightarrow [1, 2, 3], [1, 3, 2], [2, 1, 3]$
 $\Rightarrow O(n!)$
 $\Rightarrow O(n! \cdot n)$

$\Rightarrow$ $O(n!)$ is too slow

Selection sort is better

Selection Sort

$\Rightarrow$ $O(n^2)$ is better than $O(n!)$

Ex) $[8, 2, 4, 9, 3] \dots$
Step 1) $[8, 2, 4, 3, 9] \dots$
Step 2) $[2, 4, 3, 8, 9] \dots$
 $\Rightarrow O(n^2)$

$T(n) = T(n-1) + O(n)$
 $\therefore T(n) = O(n^2)$

Insertion Sort

$\Rightarrow$ $O(n^2)$ is better than $O(n!)$

Ex) $[8, 2, 4, 9, 3]$
Step 1) $[2, 8, 4, 9, 3]$
Step 2) $[2, 4, 8, 9, 3]$
Step 3) $[2, 3, 4, 8, 9]$

$\Rightarrow$ $O(n^2)$ is better than $O(n!)$

$\therefore T(n) = T(n-1) + O(n)$

$\Rightarrow T(n) = O(n^2)$

$T(1)$: 2个 10 个 2个 2个

$$\Rightarrow T(1) = O(1)$$

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n) \Rightarrow T(n) = O(n \log n)$$

递归树

$$\begin{aligned} O(n \log n) &= 2C \frac{n}{2} \log \frac{n}{2} + O(n) \\ &= 2C \frac{n}{2} \log n - 2 + O(n) \\ &= 2C \frac{n}{2} \log n - 1 + O(n) \end{aligned}$$

base: 2

$$\therefore T(n) = O(n \log n)$$

Lecture 4: Hashing

Comparison Model

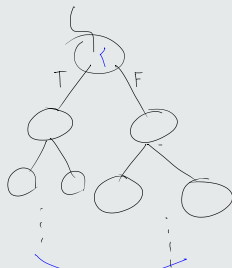
能比较 (→ 能) 在 数据 比较 时 与 数据 比较

能比较 与 能 比较 时 与 数据 比较 时 与 数据 比较

高 = 低 ≠ 可比较 的 < , > , ≥ , ≤

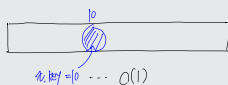
能比较 与 能 比较 时 与 数据 比较 时 与 数据 比较

min time: $O(n \log n)$

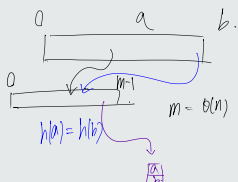


min time
→ data structure 比较 时 与 数据 比较

Direct Access Array



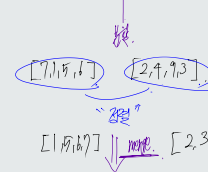
index key



Merge Sort

idea: divide and conquer.
递归 树

Ex) [7, 15, 6, 2, 4, 9, 3].



∴ [1, 2, 3, 4, 5, 6, 7, 9]

⇒ $O(n)$

√ 能 比较 与 能 比较 时 与 数据 比较 时 与 数据 比较

⇒ merge recurrence

$$T(n) = T\left(\frac{n}{2}\right) + O(n)$$

递归 树

Lecture 4: Hashing

Comparison model

$O(n \log n)$ 与 数据 比较 时 与 数据 比较

Ex) binary search.

$K < 10$?

⇒ 递归 树 比较 时 与 数据 比较

min time

(max) 递归 树

(max) "递归 树"

能 比较 与 能 比较 时 与 数据 比较 时 与 数据 比较

∴ 能 比较 与 能 比较 时 与 数据 比较 时 与 数据 比较

→ binary search 递归 树 比较 时 与 数据 比较 时 与 数据 比较

⇒ comparison model

$$\text{find}(k) = \sqrt{2} \log n$$

But.

min index ≠ comparison

Ex) 递归 树 比较 时 与 数据 比较 时 与 数据 比较

AC[10] ... 递归 树 比较 时 与 数据 比较 时 与 数据 比较

⇒ $O(1)$

Then key 比较 时 与 数据 比较 时 与 数据 比较

Direct Access Array

let $K \in [0, 1, 2, \dots, u-1]$

→ $A[K] = \text{item}$

Ex) key=2 → $A[2]$

key=9 → $A[9]$

Then find (13) is just return $A[13]$

Universal
hash function

$$h(k) = k \bmod m$$

→ 1000 2000 3000 ...
1000 2000 3000 ...
1000 2000 3000 ...

Universal
hash function

$$h_{ab}(k) = ((kab) \bmod p) \bmod m$$

$$H(p, m) = \{h_{ab}(k) \mid a, b \in \{0, \dots, p-1\} \text{ and } a \neq 0\}$$

if $a=0 \rightarrow \text{key pos.}$

Universal

$$\Pr_{h \in H} [h(k_i) = h(k_j)] \leq \frac{1}{m} \quad \forall k_i \neq k_j \in \{0, \dots, u-1\}$$

But...

Direct Access Array 2000

$$\text{Ex) } 3, 17, 891, 9999999$$

Array size ↑

∴ Direct Access Array

time: $O(1)$

space: $O(u)$

Hashing

→ 2 key 2000 2000 max
2000

Hash
function

$$h(k) : \{0, \dots, u-1\} \Rightarrow \{0, \dots, m-1\}$$

$$\text{Ex) } h(k) = k \bmod 10$$

$$17 \rightarrow 7$$

$$42 \rightarrow 2$$

$$137 \rightarrow 7$$

key 1000 2000
2000 2000
2000

Collision!

$$\text{Ex) } h(17) = 7$$

$$h(137) = 7$$

3000 key
2000 index 2000

→ $u > m \Rightarrow$ Collision 2000 2000

∴ pigeonhole principle

Chaining

→ hash table 2000
2000 2000 2000
2000 2000 2000

$$\text{Ex) } h(17) = h(137) = h(107) = 7$$

→ 7th bucket 1 chain 2000 2000

i) 2000

→ $h(k)$ 2000

ii) 2000 bucket 2000 2000

iii) chain 2000 2000

→ chain 2000 2000 2000 → hashing 2000 2000 2000

Universal (ξ

$$h_{ab}(k) = ((kab) \bmod p) \bmod m$$

Lecture 4: Hashing

Review

Data Structure	Operations $O(\cdot)$				
	Container	Static	Dynamic	Order	
	build(X)	find(k)	insert(x) delete(k)	find_min() find_max()	find_prev(k) find_next(k)
Array	n	n	n	n	n
Sorted Array	$n \log n$	$\log n$	n	1	$\log n$

- **Idea!** Want faster search and dynamic operations. Can we $\text{find}(k)$ faster than $\Theta(\log n)$?
- Answer is no (lower bound)! (But actually, yes...!?)

Comparison Model

- In this model, assume algorithm can only differentiate items via comparisons
- **Comparable items:** black boxes only supporting comparisons between pairs
- Comparisons are $<$, $\leq$, $>$, $\geq$, $=$, $\neq$, outputs are binary: True or False
- **Goal:** Store a set of n comparable items, support $\text{find}(k)$ operation
- Running time is **lower bounded** by # comparisons performed, so count comparisons!

Decision Tree

- Any algorithm can be viewed as a **decision tree** of operations performed
- An internal node represents a **binary comparison**, branching either True or False
- For a comparison algorithm, the decision tree is binary (draw example)
- A leaf represents algorithm termination, resulting in an algorithm **output**
- A **root-to-leaf path** represents an **execution of the algorithm on some input**
- Need at least **one leaf for each algorithm output**, so search requires $\geq n + 1$ leaves

∴ not in there.

Comparison Search Lower Bound

- What is worst-case running time of a comparison search algorithm?
- running time $\geq$ # comparisons $\geq$ max length of any root-to-leaf path $\geq$ height of tree
- What is minimum height of any binary tree on $\geq n$ nodes?
- Minimum height when binary tree is complete (all rows full except last)
- Height $\geq \lceil \lg(n+1) \rceil - 1 = \Omega(\log n)$, so running time of any comparison sort is $\Omega(\log n)$
- Sorted arrays achieve this bound! Yay!
- More generally, height of tree with $\Theta(n)$ leaves and max branching factor b is $\Omega(\log_b n)$
- To get faster, need an operation that allows super-constant $\omega(1)$ branching factor. How??

Direct Access Array

- Exploit Word-RAM $O(1)$ time random access indexing! Linear branching factor!
- **Idea!** Give item **unique** integer key k in $\{0, \dots, u-1\}$, store item in an array at index k
- Associate a meaning with each index of array
- If keys fit in a machine word, i.e. $u \leq 2^w$, worst-case $O(1)$ find/dynamic operations! Yay!
- 6.006: assume input numbers/strings fit in a word, unless length explicitly parameterized
- Anything in computer memory is a binary integer, or use (static) 64-bit address in memory
- But space $O(u)$, so really bad if $n \ll u$:(
- **Example:** if keys are ten-letter names, for one bit per name, requires $26^{10} \approx 17.6$ TB space
- How can we use less space?

Hashing

- **Idea!** If $n \ll u$, map keys to a smaller range $m = \Theta(n)$ and use smaller direct access array
- **Hash function:** $h(k) : \{0, \dots, u-1\} \rightarrow \{0, \dots, m-1\}$ (also hash map)
- Direct access array called **hash table**, $h(k)$ called the **hash** of key k
- If $m \ll u$, no hash function is injective by pigeonhole principle

- Always exists keys a, b such that $h(a) = h(b) \rightarrow$ **Collision!** :(
- Can't store both items at same index, so where to store? Either:
 - store somewhere else in the array (**open addressing**)
 - * complicated analysis, but common and practical
 - store in another data structure supporting dynamic set interface (**chaining**)

Chaining

- **Idea!** Store collisions in another data structure (a chain)
- If keys roughly evenly distributed over indices, chain size is $n/m = n/\Omega(n) = O(1)!$
- If chain has $O(1)$ size, all operations take $O(1)$ time! Yay!
- If not, many items may map to same location, e.g. $h(k) = \text{constant}$, chain size is $\Theta(n)$:(
- Need good hash function! So what's a good hash function?

Hash Functions

Division (bad): $h(k) = (k \bmod m)$

- Heuristic, good when keys are uniformly distributed!
- m should avoid symmetries of the stored keys
- Large primes far from powers of 2 and 10 can be reasonable
- Python uses a version of this with some additional mixing
- If $u \gg n$, every hash function will have some input set that will create $O(n)$ size chain
- **Idea!** Don't use a fixed hash function! Choose one randomly (but carefully)!

Universal (good, theoretically):

$$h_{ab}(k) = ((ak + b) \bmod p) \bmod m$$

- Hash Family $\mathcal{H}(p, m) = \{h_{ab} \mid a, b \in \{0, \dots, p-1\} \text{ and } a \neq 0\}$
- Parameterized by a fixed prime $p > u$, with a and b chosen from range $\{0, \dots, p-1\}$
- $\mathcal{H}$ is a **Universal** family: $\Pr_{h \in \mathcal{H}} \{h(k_i) = h(k_j)\} \leq 1/m \quad \forall k_i \neq k_j \in \{0, \dots, u-1\}$
- Why is universality useful? Implies short chain lengths! (in expectation)
- X_{ij} indicator random variable over $h \in \mathcal{H}$: $X_{ij} = 1$ if $h(k_i) = h(k_j)$, $X_{ij} = 0$ otherwise
- Size of chain at index $h(k_i)$ is random variable $X_i = \sum_j X_{ij}$
- Expected size of chain at index $h(k_i)$

$$\begin{aligned} \mathbb{E}_{h \in \mathcal{H}} \{X_i\} &= \mathbb{E}_{h \in \mathcal{H}} \left\{ \sum_j X_{ij} \right\} = \sum_j \mathbb{E}_{h \in \mathcal{H}} \{X_{ij}\} = 1 + \sum_{j \neq i} \mathbb{E}_{h \in \mathcal{H}} \{X_{ij}\} \\ &= 1 + \sum_{j \neq i} (1) \Pr_{h \in \mathcal{H}} \{h(k_i) = h(k_j)\} + (0) \Pr_{h \in \mathcal{H}} \{h(k_i) \neq h(k_j)\} \\ &\leq 1 + \sum_{j \neq i} 1/m = 1 + (n-1)/m \end{aligned}$$

- Since $m = \Omega(n)$, load factor $\alpha = n/m = O(1)$, so $O(1)$ **in expectation!**

Dynamic

- If n/m far from 1, rebuild with new randomly chosen hash function for new size m
- Same analysis as dynamic arrays, cost can be **amortized** over many dynamic operations
- So a hash table can implement dynamic set operations in expected amortized $O(1)$ time! :)

Data Structure	Operations $O(\cdot)$				
	Container	Static	Dynamic	Order	
	build(x)	find(k)	insert(x) delete(k)	find_min() find_max()	find_prev(k) find_next(k)
Array	n	n	n	n	n
Sorted Array	$n \log n$	$\log n$	n	1	$\log n$
Direct Access Array	u	1	1	u	u
Hash Table	$n_{(e)}$	$1_{(e)}$	$1_{(a)(e)}$	n	n

MIT OpenCourseWare
<https://ocw.mit.edu>

6.006 Introduction to Algorithms
Spring 2020

For information about citing these materials or our Terms of Use, visit: <https://ocw.mit.edu/terms>